

AI-Based Resume Screening System

This notebook demonstrates the complete workflow used in the AI-Based Resume Screening System project. It imports the project modules, preprocesses resume and job description text, generates TF-IDF features, computes similarity scores, extracts skills, ranks candidates, and displays the result table.

1. Import Required Libraries

```
In [1]: from pathlib import Path

import pandas as pd

from src.parser import extract_text
from src.preprocess import preprocess_text
from src.vectorizer import TextVectorizer
from src.matcher import compute_similarity_scores
from src.skill_extractor import compare_skills
from src.ranker import rank_resumes
```

2. Define Job Description

You can replace this sample job description with any role description.

```
In [2]: job_description = """
We are looking for a Python developer with SQL, machine learning,
data analysis, NLP, and Streamlit experience. The candidate should
understand model development, dashboards, and Git.
"""

print(job_description)
```

We are looking for a Python developer with SQL, machine learning, data analysis, NLP, and Streamlit experience. The candidate should understand model development, dashboards, and Git.

3. Load Resume Files

Place resumes inside `data/resumes/`. Supported formats are PDF, DOC, DOCX, and TXT.

```
In [3]: resume_folder = Path("data/resumes")
resume_paths = [
    path for path in resume_folder.glob("*")
    if path.is_file() and path.suffix.lower() in {".pdf", ".doc", ".docx", ".txt"}
]

resume_paths
```

Out[3]: []

4. Extract Text from Resumes

```
In [4]: file_names = []
resume_texts = []

for path in resume_paths:
    text = extract_text(path)
    file_names.append(path.name)
    resume_texts.append(text)

print(f"Loaded {len(resume_texts)} resume(s)")
```

Loaded 0 resume(s)

5. Demo Data if No Resume Files Are Available

This cell allows the notebook to run even before real resumes are added.

```
In [5]: if not resume_texts:
    file_names = ["candidate_python.txt", "candidate_design.txt"]
    resume_texts = [
        "Python developer with SQL, machine learning, pandas, NLP, Streamlit, and",
        "Graphic designer with branding, illustration, Photoshop, and portfolio",
    ]

    file_names
```

Out[5]: ['candidate_python.txt', 'candidate_design.txt']

6. Preprocess Resume and Job Description Text

```
In [6]: processed_resumes = [preprocess_text(text) for text in resume_texts]
processed_jd = preprocess_text(job_description)

processed_resumes[:2], processed_jd
```

Out[6]: (['python developer sql machine learn panda natural language process streamlit
git experience.',
'graphic designer brand illustration photoshop portfolio presentation experie
nce.'],
'we look python developer sql machine learn data analysi natural language proc
ess streamlit experience. candidate should understand model development dashboa
rd git.')

7. Generate TF-IDF Features

```
In [7]: vectorizer = TextVectorizer(mode="tfidf")
vectors = vectorizer.vectorize(processed_resumes + [processed_jd])

resume_vectors = vectors[:-1]
jd_vector = vectors[-1]
```

```
vectors.shape
```

```
Out[7]: (3, 60)
```

8. Compute Similarity Scores

```
In [8]: scores = compute_similarity_scores(resume_vectors, jd_vector)
scores
```

```
Out[8]: [0.4536375458295393, 0.01598650730421607]
```

9. Extract Matched and Missing Skills

```
In [9]: matched_skills = []
missing_skills = []

for resume_text in resume_texts:
    matched, missing = compare_skills(resume_text, job_description)
    matched_skills.append(matched)
    missing_skills.append(missing)

list(zip(file_names, matched_skills, missing_skills))

Out[9]: [('candidate_python.txt',
    {'machine learning', 'nlp', 'python', 'sql', 'streamlit'},
    {'data analysis'}),
    ('candidate_design.txt',
    set(),
    {'data analysis', 'machine learning', 'nlp', 'python', 'sql', 'streamlit'})]
```

10. Rank Resumes

```
In [10]: results = rank_resumes(
    file_names=file_names,
    scores=scores,
    matched_skills=matched_skills,
    missing_skills=missing_skills,
    threshold=60,
)

results

Out[10]: [ResumeResult(file_name='candidate_python.txt', score=0.4536375458295393, match
_percentage=45.36, matched_skills=['machine learning', 'nlp', 'python', 'sql',
'streamlit'], missing_skills=['data analysis'], status='Review'),
    ResumeResult(file_name='candidate_design.txt', score=0.01598650730421607, matc
h_percentage=1.6, matched_skills=[], missing_skills=['data analysis', 'machine
learning', 'nlp', 'python', 'sql', 'streamlit'], status='Review')]
```

11. Display Final Ranked Table

```
In [11]: ranked_df = pd.DataFrame([
    {
```

```

        "Rank": index,
        "Candidate": result.file_name,
        "Match Score": f"{result.match_percentage}%",
        "Status": result.status,
        "Extracted Skills": ", ".join(result.matched_skills) or "-",
        "Missing Skills": ", ".join(result.missing_skills) or "-",
    }
    for index, result in enumerate(results, start=1)
])

```

ranked_df

Out[11]:

	Rank	Candidate	Match Score	Status	Extracted Skills	Missing Skills
0	1	candidate_python.txt	45.36%	Review	machine learning, nlp, python, sql, streamlit	data analysis
1	2	candidate_design.txt	1.6%	Review	-	data analysis, machine learning, nlp, python, ...

12. Optional Sentence Transformer Model

Install `requirements-advanced.txt` before running this cell. It gives better semantic matching but may need model download access.

```

In [12]: # Optional advanced mode:
# vectorizer = TextVectorizer(
#     mode="sentence-transformer",
#     model_name="all-MiniLM-L6-v2",
# )
# vectors = vectorizer.vectorize(processed_resumes + [processed_jd])
# scores = compute_similarity_scores(vectors[:-1], vectors[-1])

```

13. Run the Streamlit App

Run this command in the VS Code terminal to open the web interface.

```

In [13]: # streamlit run app.py

```